

作業ログ

報告者：佐藤 夏美 (25G1062)

報告日：2026 年 6 月 10 日

プロジェクト名：受動的環境介入によるメンタルヘルス支援：大学空間における自動演奏装置の提案

1 進捗サマリー (要約)

- 全体進捗率：90% (クラリネット音色設計) , 50% (i2c 通信実装)
- マイルストーン達成状況：進行中
- 主要成果：
 - クラリネットらしい音色作成の追求で、ビブラート・音程連動 LPF を追加
 - i2c 通信の実装に着手。設計書に沿った同期プロトコル (master/slave) を製作中
- 重大課題 (影響度：高)：
 - -

2 作業ログ (詳細トラッキング)

作業項目	開始	完了予定	状態	進捗率	依存関係	コメント
音色制作	5/27	5/29	一旦完了	90%	第 6 週の音色	音程連動 LPF (freq×8, 上限 3000Hz)・ビブラート速度調整 (3.0Hz)
i2c (基礎)	5/30	6/2	完了	100%	-	ネットの i2c 通信を参考に 2 台で文字列送受信を確認 (test)
i2c (本実装)	6/3	6/17	進行中	50%	チームの同期方針	test の送受信を土台に、CONFIG/START/SYNC の同期プロトコルを実装。1 台構成で動作確認中

3 ガントチャート

カテゴリ	WBS	作業内容	担当	期間	第4週	第5週	第6週	第7週	第8週	第9週	第10週	第11週
ハードウェア	1											
	1.1	機材調達・Arduino×5動作確認	全員	第4～5週								
	1.2	通信回路(I2C配線)・音声出力回路構築	全員	第5～6週								
	1.3	筐体制作・実装	全員	第6～7週								
	1.4	ベビーメリー風回転装置製作(サーボ+シャフト+アーム+星飾り)	全員	第7～8週								
	1.5	ロードセルの反応検証	全員	第8週								
ソフトウェア	2											
	2.1	楽器別倍音比率の調査・データ化	全員	第4～5週								
	2.1.1	シリアル通信疎通確認(115200bps)	山口	第6週								
	2.1.2	フルート音色作成(倍音+ADSR)	山口	第6～8週								
	2.1.3	クラリネット音色作成(奇数倍音+LPF)	佐藤	第6～8週								
	2.1.4	オルガン音色作成(加算合成+ボイス管理)	成毛	第6～9週								
	2.1.5	ドラム音色作成(ノイズバースト+短エンベロープ)	成毛	第7～8週								
	2.1.6	Serial制御ロジック実装(5台同時受信・パート振り分け)	成毛	第7～9週								
	2.2	センサ入力・BPM変換処理(EMA+デッドバンド+×10整数化)	成毛	第5～7週								
	2.2.1	BPM変化制限ロジック(1小節最大10BPM)	成毛	第6～7週								
	2.2.2	CONFIG送信処理(起動時 entry_offset配布)	佐藤・山口	第6～7週								

図1 ガントチャート：予定1

カテゴリ	WBS	作業内容	担当	期間	第4週	第5週	第6週	第7週	第8週	第9週	第10週	第11週
ソフトウェア	2.2.3	SYNCマーカ生成・送信処理(小節頭配信)	佐藤・山口	第7～8週								
	2.2.4	START/STOP制御処理	佐藤・山口	第7～8週								
	2.2.5	サーボ制御・状態管理実装(IDLE→PLAYING→FINISHED)	佐藤・山口	第7～8週								
	2.3	共通ライブラリ整備(PROGMEM+I2Cフレーム)	成毛	第5～6週								
	2.3.1	ISR受信・デコード処理(onReceive)	佐藤・山口	第6～7週								
	2.3.2	PLL補正アルゴリズム(補正0.3+50msスナップ)	佐藤・山口	第7～8週								
	2.3.3	音符スケジューリング(PROGMEM→Serial)	各楽器担当	第6～7週								
	2.3.4	輪唱位置管理(my_local_bar計算)	各楽器担当	第7～9週								
統合・評価	3											
	3.1	単体・同期精度テスト(V1聴取+V2 FFT比較)	全員	第8～9週								
	3.2	全体結合演奏テスト(5台接続・12小節完走)	全員	第9～10週								
	3.3	可変テンポ動作検証(誤差分散5%以内)	全員	第10週								
	3.4	最終発表・報告(アンケート集計)	全員	第10～11週								

図2 ガントチャート：予定2

カテゴリ	WBS	作業内容	担当	期間	第4週	第5週	第6週	第7週	第8週	第9週	第10週	第11週
ハードウェア	1											
	1.1	機材調達・Arduino×5動作確認	全員	第4～5週								
	1.2	通信回路(I2C配線)・音声出力回路構築	全員	第5～6週								
	1.3	筐体制作・実装	全員	第6～7週								
	1.4	ベビーメリー風回転装置製作(サーボ+シャフト+アーム+星飾り)	全員	第7～8週								
	1.5	ロードセルの反応検証	全員	第8週								
ソフトウェア	2											
	2.1	楽器別倍音比率の調査・データ化	全員	第4～5週								
	2.1.1	シリアル通信疎通確認(115200bps)	山口	第6週								
	2.1.2	フルート音色作成(倍音+ADSR)	山口	第6～8週								
	2.1.3	クラリネット音色作成(奇数倍音+LPF)	佐藤	第6～8週								
	2.1.4	オルガン音色作成(加算合成+ボイス管理)	成毛	第6～9週								
	2.1.5	ドラム音色作成(ノイズバースト+短エンベロープ)	成毛	第7～8週								
	2.1.6	Serial制御ロジック実装(5台同時受信・パート振り分け)	成毛	第7～9週								
	2.2	センサ入力・BPM変換処理(EMA+デッドバンド×10整数化)	成毛	第5～7週								
	2.2.1	BPM変化制限ロジック(1小節最大10BPM)	成毛	第6～7週								
	2.2.2	CONFIG送信処理(起動時 entry_offset配布)	佐藤・山口	第6～7週								

図3 ガントチャート：現在1

カテゴリ	WBS	作業内容	担当	期間	第4週	第5週	第6週	第7週	第8週	第9週	第10週	第11週
ソフトウェア	2.2.3	SYNCマーカ生成・送信処理(小節頭配信)	佐藤・山口	第7～8週								
	2.2.4	START/STOP制御処理	佐藤・山口	第7～8週								
	2.2.5	サーボ制御・状態管理実装(IDLE→PLAYING→FINISHED)	佐藤・山口	第7～8週								
	2.3	共通ライブラリ整備(PROGMEM+I2Cフレーム)	成毛	第5～6週								
	2.3.1	ISR受信・デコード処理(onReceive)	佐藤・山口	第6～7週								
	2.3.2	PLL補正アルゴリズム(補正0.3+50msスナップ)	佐藤・山口	第7～8週								
	2.3.3	音符スケジューリング(PROGMEM→Serial)	各楽器担当	第6～7週								
	2.3.4	輪唱位置管理(my_local_bar計算)	各楽器担当	第7～9週								
統合・評価	3											
	3.1	単体・同期精度テスト(V1聴取+V2 FFT比較)	全員	第8～9週								
	3.2	全体結合演奏テスト(5台接続・12小節完走)	全員	第9～10週								
	3.3	可変テンポ動作検証(誤差分散5%以内)	全員	第10週								
	3.4	最終発表・報告(アンケート集計)	全員	第10～11週								

図4 ガントチャート：現在2

4 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
クラリネットとフルートの違い	それぞれの楽器の特徴的な鳴り方	中	中	ビブラートやノイズの値調整	6/3	停滞
LPF カットオフ設計の硬直性	カットオフの設定が固定になっていて汎用性に欠ける	中	中	フルートの設計を参考に「基音 × N (例: N=7~9)」の形でノートに追従させる方法に変更	5/28	解決

※影響度＝プロジェクトへのインパクト ※優先度＝対応の緊急性

5 AI 利用状況と検証（再現性重視）

5.1 利用内容

- ・利用目的：レビュー・実装（音色のブラッシュアップと、i2c 通信実装上の確認）。
- ・入力（プロンプト要旨）：
 - 仕様書要素まで作った音色を、もっと本物のクラリネットに近づけたい、木管らしい響きや吹き始めの不安定感を足したい。
 - 受信割り込みではフラグを立てるだけにしてメインループで処理させたが、割り込みがバッファに書き込む途中でメインループが読むと値が混ざる。割り込みとメインループ間でデータを安全に渡すにはどうすべきか。
 - 起動時に CONFIG → START を続けて送ると、スレーブが START を取りこぼすことがある。連続するコマンドの間隔はどうすべきか。
- ・出力概要：
 - ビブラートやピッチドロップ（アタック時に音程がずり下がる効果）の提案と解説。
 - 共有変数に volatile を付け、読み出しは noInterrupts()～interrupts() で囲んで短時間でコピーする。
 - スレーブの処理に間があるので、CONFIG の後に数ミリ秒 delay を入れてから START を送れば取りこぼさない。

5.2 検証方法

※第三者が再現できるレベルで記述

- ・検証手法：

- FFT 解析 (実測との比較).
- 実音源との聴感比較.
- 実機でのテスト.
- 公式ドキュメント・定番手法との照合.
- 参照資料：
 - 実測に用いたクラリネット音源：MTG (Music Technology Group, Universitat Pompeu Fabra) , [freesound.org](https://freesound.org/people/MTG/sounds/248359/) (音源番号 248359)
<https://freesound.org/people/MTG/sounds/248359/>
 - Processing 公式リファレンス：The Processing Foundation
<https://processing.org/reference/>
 - Minim 公式ドキュメント
<https://code.compartmental.net/minim/>
 - Arduino 公式 Wire ライブラリ解説
<https://docs.arduino.cc/learn/communication/wire/>
 - MergeCells 「Arduino を 30 分で I2C 通信する」 Qiita
<https://qiita.com/MergeCells/items/20c3c1a0adfb222a19cd>
- 検証手順：
 - ブラッシュアップ後の合成音を FFT 解析し追加要素を入れても、奇数倍音優勢の構造が崩れていないか確認する.
 - 合成音とクラリネット実音源を並べて比較し、より自然になったか聴いて確認する.
 - ビブラート幅・ノイズ音量・LPF カットオフを聴きながら調整する.
 - 提案内容を実装前後で比較.
 - 実装内容読み取って、公式ドキュメントや定番手法と照らし合わせて確認する.

5.3 ファクトチェック結果

- 一致点：
 - 表現要素を追加しても FFT 解析で奇数倍音優勢の構造が保たれ、ノイズも仕様書の帯域に収まっていた.
 - 合成音とクラリネット実音源の比較で、ビブラート追加後の方が実音源に近い響きになった.
 - volatile + 割り込み停止でコピーすると、受信値が混ざる症状が出なくなった.
 - CONFIG と START の間に待ちを入れると、両方とも確実に処理された.
- 不一致・疑義：
 - AI 提案はビブラートが過大で不自然.
 - ピッチドロップはクラリネットの音の出方とは違い、機械音のような不自然な効果になった.

- volatile だけで十分とも示唆されたが、連続コピー中の競合は防げず、noInterrupts()での保護が必要と判断。
 - 最終判断：
 - 音色作成, i2c とともに一部採用。
 - 根拠：
 - 実音源との聴感比較で、過大だった要素を調整した結果がより自然と判断できたため。
 - 設計は自分で決め、AI には実装上の確認をただけで、得た内容も実機と公式ドキュメントで確認したため。
-

6 次回計画

- 目標：
 - Arduino から届く 5 バイトの音符データを受信して発音する処理を実装し、Processing との連携を進める
 - i2c 通信の作成
 - 達成基準：
 - 受信した音符データで、作成した音色が正しく発音されること
 - i2c 通信で Arduino と Processing がデータの送受信ができること
 - 期限：
 - 2026 年 6 月 17 日
-

7 その他

- ビブラート・音程連動 LPF は仕様書には無い追加要素であり、音質向上のため導入した
-

8 付録

- 添付資料：
 - ソースコード, 資料：<https://github.com/7nanasi/hackathon>
- 添付範囲：全体